



# Semántica de lenguajes de programación

Fundamentos de Lenguajes de Programación – IIC2560

Yadran Eterovic S. (yadran@uc.cl)



“...There's a sign on the wall, but she wants to be sure  
'Cause you know sometimes words have two meanings...”

Estudio de los lenguajes de programación:

- estudio de la sintaxis
- estudio de la **semántica**

La forma de las expresiones, sentencias y unidades de un programa

P.ej., la sintaxis de la sentencia **while** en Java es  
`while (boolean_expr) statement`

El significado de esas expresiones, sentencias y unidades

P.ej., la semántica de la sentencia **while** es que  
Cuando el valor actual de *boolean\_expr* es verdadero,  
la sentencia *statement*, subordinada, es ejecutada  
... y el control vuelve (implícitamente) a *boolean\_expr*  
para repetir el proceso.

Si *boolean\_expr* es falsa, entonces el control es transferido a la sentencia que sigue al **while**

La descripción de la **semántica** de un lenguaje de programación es más compleja que la de la sintaxis:

- problemas técnicos
- la necesidad de mediar entre exactitud y flexibilidad

Se necesita una descripción precisa, no ambigua de lo que debe esperarse de cada constructo sintácticamente correcto

... pero esta descripción no debe anticipar elecciones propias de la implementación y que no son parte del lenguaje

La descripción de la **semántica** de un lenguaje de programación es más compleja que la de la sintaxis:

- problemas técnicos
- la necesidad de mediar entre exactitud y flexibilidad

Se necesita una descripción precisa, no ambigua de lo que debe esperarse de cada constructo sintácticamente correcto

... pero esta descripción no debe anticipar elecciones propias de la implementación y que no son parte del lenguaje

Debe poder saberse antes de la ejecución del programa, e independientemente de la arquitectura, **qué va a pasar a la hora de la ejecución**

Debe **permitir diferentes implementaciones del lenguaje**, todas correctas con respecto a la semántica

No es difícil lograr exactitud:

- la semántica se puede describir usando un compilador específico y una arquitectura específica
- el significado “oficial” de un programa corresponde a su ejecución en esta arquitectura después de ser traducido por este compilador

... pero a costa de flexibilidad:

- existe sólo una implementación

... pero, ¿hasta qué nivel de detalle esta implementación es normativa?

No es difícil lograr exactitud:

- la semántica se puede describir usando un compilador específico y una arquitectura específica
- el significado “oficial” de un programa corresponde a su ejecución en esta arquitectura después de dser traducido por este compilador

... pero a costa de flexibilidad:

- existe sólo una implementación

... pero, ¿hasta qué nivel de detalle esta implementación es normativa?

... y todas las otras que puede haber son totalmente equivalentes

¿Es el tiempo de ejecución de un programa parte de la definición?

Cuando la tecnología de implementación de la arquitectura física cambie, ¿cambia la semántica?

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad



Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

En **semántica operacional**, el significado de un programa es dado por el comportamiento de un computador abstracto —el intérprete del lenguaje “Análisis de casos exhaustivo”  
Hay gran variedad de técnicas operacionales

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

En **semántica operacional**, el significado de un programa es dado por el comportamiento de un computador abstracto —el intérprete del lenguaje “Análisis de casos exhaustivo”

Hay gran variedad de técnicas operacionales

En **semántica axiomática**, el significado de un programa es dado por las transformaciones que la ejecución de los comandos produce en el estado, caracterizado por fórmulas de lógica de predicados

Los axiomas son fórmulas a priori verdaderas

Especifica lo que puede ser demostrado acerca del programa

### Semántica operacional

1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:

- Cada constructo debe tener un significado obvio no ambiguo

2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

## Semántica operacional

- 1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:
  - Cada constructo debe tener un significado obvio no ambiguo
- 2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

### *C Statement*

```
for (expr1; expr2; expr3) {  
    ...  
}
```

Ej. usado en textos de programación y manuales de referencia de lenguajes

### *Meaning*

```
    expr1;  
loop: if expr2 == 0 goto out  
    ...  
    expr3;  
    goto loop  
out: ...
```

## Semántica operacional

- 1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:
  - Cada constructo debe tener un significado obvio no ambiguo
- 2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

```
ident = var
ident = ident + 1
ident = ident - 1
goto label
if var relop var goto label
ident = var bin_op var
ident = un_op var
```

... en que *relop* es uno de =, <>, >, <, >=, <=  
... *ident* es un identificador  
... *var* es un identificador o una constante  
... *bin\_op* es un operador aritmético binario  
... *un\_op* es un operador unario

```

ident = var
ident = ident + 1
ident = ident - 1
goto label
if var relop var goto label

```

```

ident = var bin_op var
ident = un_op var

```

```

for (count1 = 0, count2 = 1.0;
      count1 <= 10 && count2 <= 100.0;
      sum = ++count1 + count2, count2 *= 2.5);

```

... en que *relop* es uno de =, <>, >, <, >=, <=

... *ident* es un identificador

... *var* es un identificador o una constante

... *bin\_op* es un operador aritmético binario

... *un\_op* es un operador unario

```

count1 = 0
count2 = 1.0
loop:
  if count1 > 10 goto out
  if count2 > 100.0 goto out
  count1 = count1 + 1
  sum = count1 + count2
  count2 = count2 * 2.5
  goto loop
out: ...

```

```

ident = var
ident = ident + 1
ident = ident - 1
goto label
if var relop var goto label
ident = var bin_op var
ident = un_op var

```

... en que *relop* es uno de =, <>, >, <, >=, <=

... *ident* es un identificador

... *var* es un identificador o una constante

... *bin\_op* es un operador aritmético binario

... *un\_op* es un operador unario

```

sum = 0;
indat = Int32.Parse(Console.ReadLine());
while (indat >= 0) {
    sum += indat;
    indat = Int32.Parse(Console.ReadLine());
}

value = Int32.Parse(Console.ReadLine());
do {
    value /= 10;
    digits ++;
} while (value > 0);

```

```

loop:
    if control_expression is false goto out
    [loop body]
    goto loop
out: ...

```

```

loop:
    [loop body]
    if control_expression is true goto loop

```



### Semántica axiomática

Basada en una *lógica de programación*:

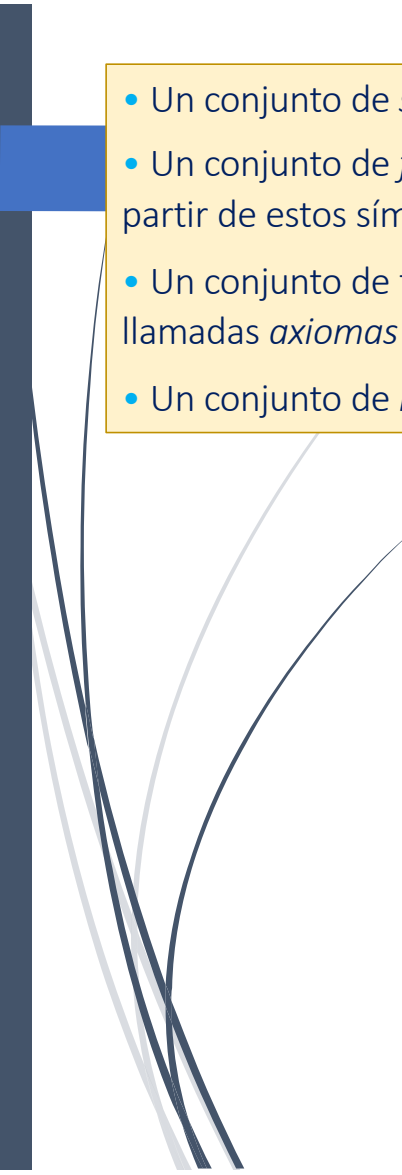
- **Un sistema lógico formal** que facilita hacer afirmaciones precisas acerca de la ejecución de un programa

## Semántica axiomática

Basada en una *lógica de programación*:

- Un **sistema lógico formal** que facilita hacer afirmaciones precisas acerca de la ejecución de un programa

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

- 
- Un conjunto de *símbolos*
  - Un conjunto de *fórmulas* construidas a partir de estos símbolos
  - Un conjunto de fórmulas distinguidas llamadas *axiomas*
  - Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

Fórmulas

Cada  $H_i$  es un **hipótesis**

$C$  es una **conclusión**

$$\frac{H_1, H_2, \dots, H_n}{C}$$

Si las hipótesis son verdaderas, entonces podemos inferir que la conclusión es verdadera

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

Fórmulas

Cada  $H_i$  es un **hipótesis**

$C$  es una **conclusión**

$$\frac{H_1, H_2, \dots, H_n}{C}$$

Si las hipótesis son verdaderas, entonces podemos inferir que la conclusión es verdadera

Una **demostración** es una secuencia de líneas, c/u de las cuales es un axioma o puede ser derivada de las líneas anteriores mediante la aplicación de una regla de inferencia

## Lógica de programación

Un sistema lógico formal que permite establecer y demostrar propiedades de programas

### Predicados, llaves y comandos del lenguaje

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Comando o lista de comandos

Triple: { P } S { Q }

Su interpretación caracteriza la relación entre P y Q, y S

Predicados que especifican las relaciones entre los valores de las variables del programa

Comando o lista de comandos

Triple:  $\{P\} S \{Q\}$

Predicados —*aseveraciones*— que especifican las relaciones entre los valores de las variables del programa

Su *interpretación* caracteriza la relación entre  $P$  y  $Q$ , y  $S$

Precondición

Postcondición

El triple  $\{P\} S \{Q\}$  es verdadero si, siempre que la ejecución de  $S$  empiece en un estado que satisface  $P$  y la ejecución de  $S$  termina, entonces el estado resultante satisface  $Q$

Corrección parcial:  
“... y la ejecución de  $S$  termina ...”

Si la lógica es correcta (con respecto a la interpretación), entonces todos los teoremas van a ser afirmaciones verdaderas acerca del dominio

Esto requiere que:

- todos los axiomas sean correctos
- todas las reglas de inferencia sean correctas

Correspondan a verdadero

La conclusión corresponde a verdadero, suponiendo que todas las hipótesis corresponden a verdadero



Si la lógica es correcta (con respecto a la interpretación), entonces todos los teoremas van a ser afirmaciones verdaderas acerca del dominio

Esto requiere que:

- todos los axiomas sean correctos
- todas las reglas de inferencia sean correctas

Correspondan a verdadero

La conclusión corresponde a verdadero, suponiendo que todas las hipótesis corresponden a verdadero

E.g., la fórmula (o triple)

$\{x = 0\} \ x \leftarrow x+1; \ \{x = 1\}$

... debe ser un teorema

... pero el triple

$\{x = 0\} \ x \leftarrow x+1; \ \{y = 1\}$

... no debería serlo

El axioma más importante de una lógica de programación, para lenguajes imperativos, es el **axioma de asignación**:

$$\{ P_{x \leftarrow expr} \} x \leftarrow expr \{ P \}$$

Para que una asignación resulte en un estado que satisfaga el predicado **P**

... el estado anterior debe satisfacer **P** con la variable **x** textualmente reemplazada por la expresión *expr*

El axioma más importante de una lógica de programación, para lenguajes imperativos, es el **axioma de asignación**:

$$\{ P_{x \leftarrow expr} \} x \leftarrow expr \{ P \}$$

Describe cómo cambian los estados

Para que una asignación resulte en un estado que satisfaga el predicado **P**

... el estado anterior debe satisfacer **P** con la variable **x** textualmente reemplazada por la expresión **expr**

Especifica sustitución textual:

Reemplazar todas las ocurrencias libres de la variable **x** en el predicado **P** por la expresión **expr**

**x** es libre en **P** si no es capturada por una variable acotada con el mismo nombre en un  $\exists$  o  $\forall$

El axioma más importante de una lógica de programación, para lenguajes imperativos, es el **axioma de asignación**:

$$\{ P_{x \leftarrow expr} \} x \leftarrow expr \{ P \}$$

Describe cómo cambian los estados

Para que una asignación resulte en un estado que satisfaga el predicado  $P$

... el estado anterior debe satisfacer  $P$  con la variable  $x$  textualmente reemplazada por la expresión  $expr$

Especifica sustitución textual:

Reemplazar todas las ocurrencias libres de la variable  $x$  en el predicado  $P$  por la expresión  $expr$

$x$  es libre en  $P$  si no es capturada por una variable acotada con el mismo nombre en un  $\exists$  o  $\forall$

E.g.:

$$\{ 1 = 1 \} x \leftarrow 1 \{ x = 1 \}$$

Sin importar cuál sea el estado de partida, si se asigna 1 a  $x$  se obtiene un estado que satisface  $x = 1$

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla de  
composición

$$\frac{\{P\} S_1 \{Q\}, \{Q\} S_2 \{R\}}{\{P\} S_1; S_2 \{R\}}$$

Efecto de la ejecución de dos comandos, uno después del otro

Regla del  
comando **if**

$$\frac{\{P \wedge B\} S \{Q\}, (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{if } (B) S; \{Q\}}$$

La primera hipótesis caracteriza el efecto de ejecutar **S** cuando **B** es verdadero; la segunda, qué es verdadero cuando **B** es falso  
La conclusión combina ambos casos

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla de  
composición

$$\frac{\{P\} S_1 \{Q\}, \{Q\} S_2 \{R\}}{\{P\} S_1; S_2 \{R\}}$$

Efecto de la ejecución de dos comandos, uno después del otro

Regla del  
comando **if**

$$\frac{\{P \wedge B\} S \{Q\}, (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{ if } (B) S; \{Q\}}$$

La primera hipótesis caracteriza el efecto de ejecutar **S** cuando **B** es verdadero; la segunda, qué es verdadero cuando **B** es falso  
La conclusión combina ambos casos

```
{ true }
m ← x;
{ m = x }
if (y > m)
    m ← y;
{ (m = x ∧ m ≥ y) ∨ (m = y ∧ m > x) }
```

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla del  
comando **while**

$$\frac{\{I \wedge B\} S \{I\}}{\{I\} \text{ while}(B) S; \{I \wedge \neg B\}}$$

$$\frac{P' \Rightarrow P, \{P\} S \{Q\}, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

Requiere un *invariante* **I** —un predicado que es verdadero antes y después de cada iteración del *loop*

- si **I** y **B** son verdaderos antes de ejecutar **S**, entonces la ejecución de **S** debe hacer de nuevo verdadero a **I**
- cuando el *loop* termine, **I** seguirá siendo verdadero, pero ahora **B** será *falso*



Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla del  
comando **while**

$$\frac{\{I \wedge B\} S \{I\}}{\{I\} \text{while}(B) S; \{I \wedge \neg B\}}$$

$$\frac{P' \Rightarrow P, \{P\} S \{Q\}, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

Requiere un *invariante* **I** —un predicado que es verdadero antes y después de cada iteración del *loop*

- si **I** y **B** son verdaderos antes de ejecutar **S**, entonces la ejecución de **S** debe hacer de nuevo verdadero a **I**
- cuando el *loop* termine, **I** seguirá siendo verdadero, pero ahora **B** será *falso*

```
i ← 1;
{i = 1 ∧ (∀ j: 1 ≤ j < i: a[j] ≠ x)}
while (a[i] ≠ x)
    i ← i+1;
{(∀ j: 1 ≤ j < i: a[j] ≠ x) ∧ a[i] = x}
```

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla de  
consecuencia

$$\frac{\{I \wedge B\} S \{I\}}{\{I\} \text{ while}(B) S; \{I \wedge \neg B\}}$$


---


$$\frac{P' \Rightarrow P, \{P\} S \{Q\}, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

Permite que las precondiciones sean fortalecidas y/o las postcondiciones sean debilitadas

Las **reglas de inferencia** permiten derivar teoremas que resultan de combinar instancias del axioma de asignación:

- caracterización de los efectos de la composición de comandos y de los comandos de control **if** y **while**

Regla de  
consecuencia

$$\frac{\frac{\{I \wedge B\} S \{I\}}{\{I\} \text{ while}(B) S; \{I \wedge \neg B\}}}{P' \Rightarrow P, \{P\} S \{Q\}, Q \Rightarrow Q'}$$

$$\frac{}{\{P'\} S \{Q'\}}$$

Permite que las precondiciones sean fortalecidas y/o las postcondiciones sean debilitadas

Este triple es verdadero

$$\{x = 0\} x \leftarrow x+1; \{x = 1\}$$

... y por la regla de consecuencia, éste también

$$\{x = 0\} x \leftarrow x+1; \{x > 0\}$$

### Semántica de ejecución concurrente:

#### Los comandos **await** y **co**

Los procesos consisten en una secuencia de comandos, secuenciales —diapos anteriores— y de sincronización

La sincronización por condición hace que un proceso espere hasta que se cumpla una condición

En lugar de comandos concretos —*locks*, semáforos, monitores, etc.— usamos el comando abstracto **await**:

`< await (B) S; >`

## Semántica de ejecución concurrente:

### Los comandos **await** y **co**

Los procesos consisten en una secuencia de comandos, secuenciales —diapos anteriores— y de sincronización

La sincronización por condición hace que un proceso espere hasta que se cumpla una condición

En lugar de comandos concretos —*locks*, semáforos, monitores, etc.— usamos el comando abstracto **await**:

**`< await (B) S; >`**

**`< y >`** indican que **await** se ejecuta como una acción atómica:

- **B** es garantizadamente verdadero cuando comienza la ejecución de **S**
- ningún estado interno en **S** es visible para otros procesos

Condición (de postergación)

Secuencia de comandos secuenciales (que garantizadamente termina)

## Semántica de ejecución concurrente: Los comandos **await** y **co**

Los procesos consisten en una secuencia de comandos, secuenciales —diapos anteriores— y de sincronización

La sincronización por condición hace que un proceso espere hasta que se cumpla una condición

En lugar de comandos concretos —*locks*, semáforos, monitores, etc.— usamos el comando abstracto **await**:

$\langle \text{await } (B) S; \rangle$

$\langle y \rangle$  indican que **await** se ejecuta como una acción atómica:

- **B** es garantizadamente verdadero cuando comienza la ejecución de **S**
- ningún estado interno en **S** es visible para otros procesos

Condición (de postergación)

Secuencia de comandos secuenciales (que garantizadamente termina)

E.g.,

$\langle \text{await } (n > 0) n \leftarrow n-1; \rangle$

... posterga (al proceso que la ejecuta) hasta que **n** sea positivo; entonces decrementa **n**

→ **n** es garantizadamente positivo antes de ser decrementado

El efecto de un comando **await** es similar al de un comando **if** para el cual **B** es verdadera cuando empieza la ejecución de **S**

Regla del  
comando **await**

$$\frac{\{P \wedge B\} S \{Q\}}{\{P\} \langle \text{await } (B) S; \rangle \{Q\}}$$

Si la ejecución de **S** empieza en un estado en que **P** y **B** son verdaderos, y **S** termina, entonces **Q** será verdadero  
 → **await** produce el estado **Q** si es iniciado en el estado **P** y termina  
 ( Las reglas de inferencia no dicen nada acerca de las posibles demoras)

Regla del  
comando **if**

$$\frac{\{P \wedge B\} S \{Q\}, (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{if } (B) S; \{Q\}}$$

### Semántica de ejecución concurrente: Los comandos **await** y **co**

Un programa concurrente consiste en múltiples procesos ejecutando concurrentemente

La ejecución concurrente de  $n$  (listas de) comandos  $S_i$  se especifica mediante el comando **co**:

$$\text{co } S_1; \parallel S_2; \parallel \dots \parallel S_n; \text{ oc};$$

... en que cada (lista de) comando(s)  $S_i$  satisface el triple

$$\{ P_i \} S_i \{ Q_i \}$$



## Semántica de ejecución concurrente: Los comandos **await** y **co**

Un programa concurrente consiste en múltiples procesos ejecutando concurrentemente

La ejecución concurrente de  $n$  (listas de) comandos  $S_i$  se especifica mediante el comando **co**:

**co**  $S_1$ ; ||  $S_2$ ; || ... ||  $S_n$ ; **oc**;

... en que cada (lista de) comando(s)  $S_i$  satisface el triple

$\{ P_i \} S_i \{ Q_i \}$

Operador de  
paralelismo

Si  $S_i$  empieza en un estado que satisface  $P_i$   
y termina, entonces el estado satisfará  $Q_i$

Empiece a ejecutar cada comando  $S_i$  en paralelo  
... luego, espere a que terminen  
**co** termina una vez que todos los comandos han  
sido ejecutados

Para que la interpretación anterior (“Si  $S_i$  empieza en ...”) sea válida cuando los procesos son ejecutados concurrentemente ... los procesos deben partir en un estado que satisfaga la conjunción de los  $P_i$

Si todos los procesos terminan, entonces el estado final satisfará la conjunción de los  $Q_i$

Regla del  
comando **co**

$\{P_i\} \quad S_i \quad \{Q_i\}$  are interference free

$\{P_1 \wedge \dots \wedge P_n\}$

**co**  $S_1; // \dots // S_n; \text{ oc}$

$\{Q_1 \wedge \dots \wedge Q_n\}$

Para que la interpretación anterior (“Si  $S_i$  empieza en ...”) sea válida cuando los procesos son ejecutados concurrentemente ... los procesos deben partir en un estado que satisfaga la conjunción de los  $P_i$

Si todos los procesos terminan, entonces el estado final satisfará la conjunción de los  $Q_i$

Regla del  
comando **co**

$\{P_1\} S_1 \{Q_1\}$  are interference free

$\{P_1 \wedge \dots \wedge P_n\}$

**co**  $S_1; // \dots // S_n$  **oc**

$\{Q_1 \wedge \dots \wedge Q_n\}$

Los procesos y sus triples *no deben interferir* entre ellos

Un proceso *interfiere* con otro si ejecuta una asignación que invalida una aseveración en el otro proceso

Regla del  
comando **co**

$\{P_i\} \ S_i \ \{Q_i\}$  are interference free

$\{P_1 \wedge \dots \wedge P_n\}$

**co**  $S_1; \ // \ \dots \ // \ S_n; \ \mathbf{oc}$

$\{Q_1 \wedge \dots \wedge Q_n\}$

Los procesos y sus triples *no*  
*deben interferir* entre ellos

Un proceso *interfiere* con otro si ejecuta una asignación que invalida una aseveración en el otro proceso:

- las aseveraciones caracterizan lo que un proceso supone que es verdadero antes y después de cada comando
- si un proceso asigna a una variable compartida y consecuentemente invalida una suposición de otro proceso, entonces el triple del otro proceso no es válido

Regla del  
comando **co**

$\{P_i\} \ S_i \ \{Q_i\}$  are interference free

---

$\{P_1 \wedge \dots \wedge P_n\}$   
**co**  $S_1; \ // \ \dots \ // \ S_n; \ \mathbf{oc}$   
 $\{Q_1 \wedge \dots \wedge Q_n\}$

Los procesos y sus triples *no deben interferir* entre ellos

Un proceso *interfiere* con otro si ejecuta una asignación que invalida una aseveración en el otro proceso:

- las aseveraciones caracterizan lo que un proceso supone que es verdadero antes y después de cada comando
- si un proceso asigna a una variable compartida y consecuentemente invalida una suposición de otro proceso, entonces el triple del otro proceso no es válido

```
{x == 0}
co <x = x+1;> // <x = x+2;> oc
{x == 3}
```

E.g., ningún proceso puede suponer que **x** es 0 cuando empieza a ejecutar  
 ... si lo hace, esa aseveración se verá interferida si el otro proceso ejecuta primero

Regla del  
comando **co**

$\{P_i\} \ S_i \ \{Q_i\}$  are interference free

---

$\{P_1 \wedge \dots \wedge P_n\}$   
**co**  $S_1; \ // \ \dots \ // \ S_n; \ \mathbf{oc}$   
 $\{Q_1 \wedge \dots \wedge Q_n\}$

Los procesos y sus triples *no*  
*deben interferir* entre ellos

```
{x == 0}
co {x == 0 ∨ x == 2}
  ⟨x = x+1;⟩
  {x == 1 ∨ x == 3}
// {x == 0 ∨ x == 1}
  ⟨x = x+2;⟩
  {x == 2 ∨ x == 3}
oc
{x == 3}
```

Las aseveraciones en cada proceso toman en  
cuenta los dos posibles órdenes de ejecución  
La conjunción de las precondiciones es  $x = 0$   
... y la de las postcondiciones,  $x = 3$